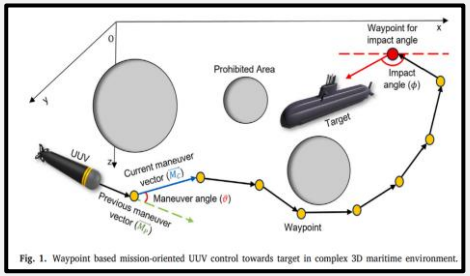


Fig. 1. Waypoint based mission-oriented UUV control towards target in complex 3D maritime environment.

현대 강화학습 자율이동체 구현 워크숍

수정된 PPO기반 무인이동체 제어

Emily Jimin Roh
Korea University, School of Electrical Engineering
Artificial Intelligence and Mobility Laboratory
emilyjroh@korea.ac.kr

목차

수정된 PPO기반 무인이동체 제어 연구 개요

수정된 PPO 이론 및 설계

수정된 PPO기반 무인이동체 제어 구현

Q&A

목차

수정된 PPO기반 무인이동체 제어 연구 개요

수정된 PPO 이론 및 설계

수정된 PPO기반 무인이동체 제어 구현

Q&A

수정된 PPO기반 무인이동체 제어 연구 개요

- 연구 개요

Ocean Engineering 320 (2025) 120242

Contents lists available at ScienceDirect

Ocean Engineering

journal homepage: www.elsevier.com/locate/oceaneng

Research paper

Autonomous mission-oriented unmanned underwater vehicle control using directional policy optimization

Emily Jimin Roh [a], Il Seok Song [b], Seunghwan Kim [b], Soohyun Park [c,*]

[a] Department of Electrical and Computer Engineering, Korea University, Seoul 02841, Republic of Korea
[b] Maritime Artificial Intelligence, LIG Nex1, Seongnam 13488, Republic of Korea
[c] Division of Computer Science, Sookmyung Women's University, Seoul 04310, Republic of Korea

ARTICLE INFO

Keywords:
Unmanned underwater vehicle (UUV)
Directional policy optimization (DPO)
Waypoint
Impact angle

ABSTRACT

With the increasing demand for scientific and military operations in marine environments, unmanned underwater vehicles (UUVs) have become essential due to their adaptability in conducting various maritime missions. Autonomous control systems further enhance this flexibility, particularly in mission-specific operations. However, much of the existing research focuses predominantly on 2D environments, often neglecting the complexities and orientation challenges that arise in real-world 3D underwater scenarios. To address these issues, this paper introduces a 5-degree-of-freedom (DOF) control approach for UUVs, aimed at improving waypoint-based path planning in 3D maritime missions through directional policy optimization (DPO). Utilizing a deep reinforcement learning (DRL) framework, the proposed method employs an efficient directional policy that accounts for the unique maneuvering characteristics of UUVs, including the maximum feasible rotation angles under dynamic constraints. The DPO algorithm enables UUVs to achieve efficient path planning by minimizing the number of waypoints required. Additionally, by considering the impact angle, which dictates the approach angle to the target, the methodology facilitates a mission execution strategy that minimizes the UUV's exposure, thus enhancing stealth during military operations.

E. J. Roh, I. S. Song, S. Kim, S. Park, "Autonomous mission-oriented unmanned underwater vehicle control using directional policy optimization," *Ocean Engineering*, vol. 320, pp. 120242, 2025.

목표)
수정된 PPO기반 목표물 타격을 위한 무인이동체 제어를 목표로 함. 이때 무인이동체의 특성에 따라 PPO의 학습 환경을 설정하는 것이 중요함.

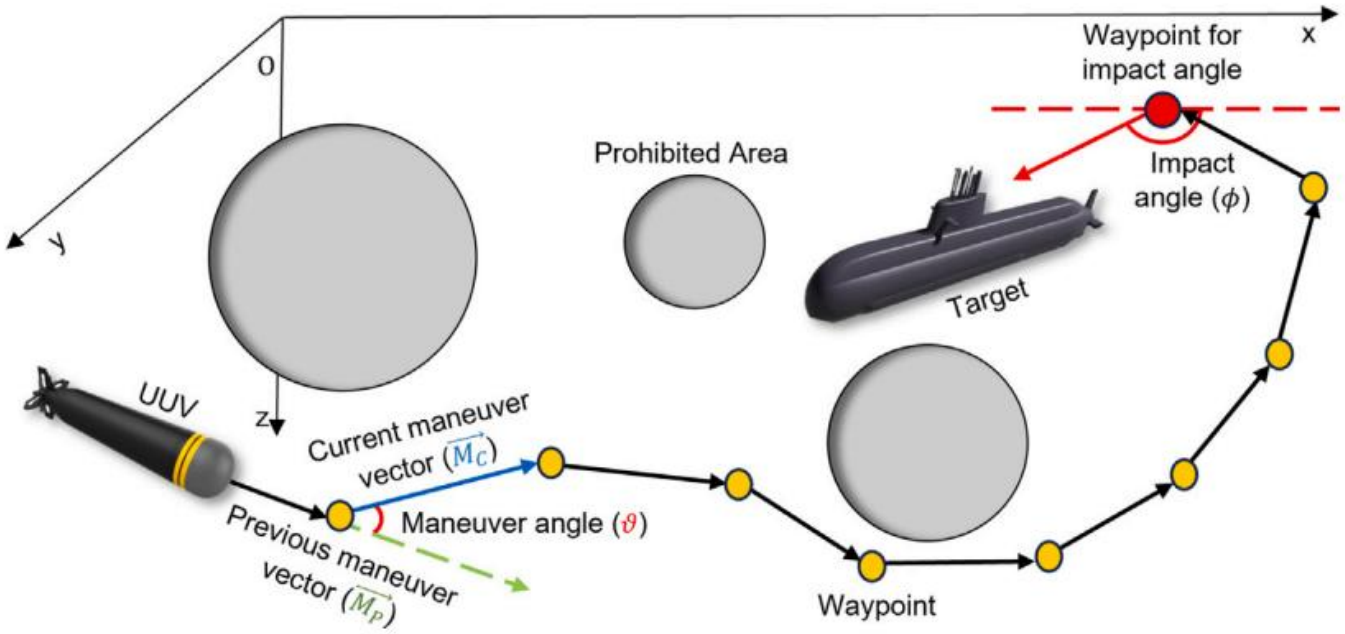


Fig. 1. Waypoint based mission-oriented UUV control towards target in complex 3D maritime environment.

목차

수정된 PPO기반 무인이동체 제어 연구 개요

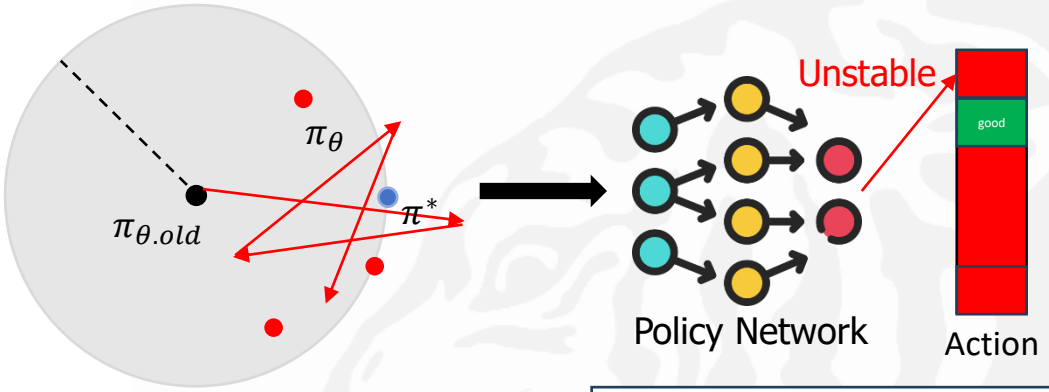
수정된 PPO 이론 및 설계

수정된 PPO기반 무인이동체 제어 구현

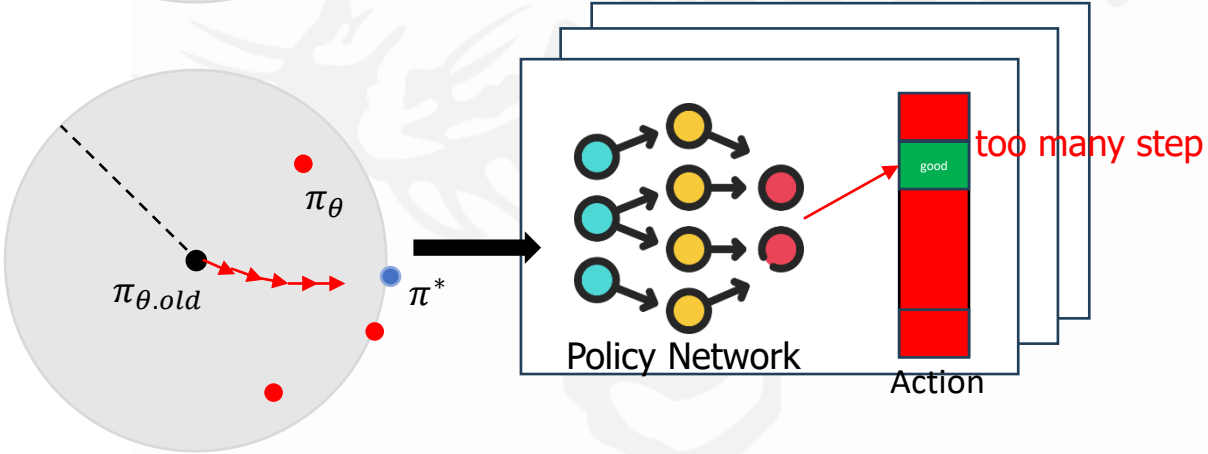
Q&A

- **Proximal Policy Optimization (PPO)**
  - History & Basic information
    - 2017년, John Schulman이 미국 인공지능 회사 OpenAI에서 개발한 강화 학습 알고리즘으로, 현재는 많이 사용되는 강화학습 알고리즘 중 하나로 자리함.
    - PPO는 다른 정책 기울기(policy gradient) 방법에 비해 더 나은 성능을 보임.
    - 신뢰 영역 정책 최적화(TRPO)와 비교하여 구현이 상대적으로 간단함.
  - Background
    - 정책 기울기 방법의 불안정성
    - 데이터 비효율성

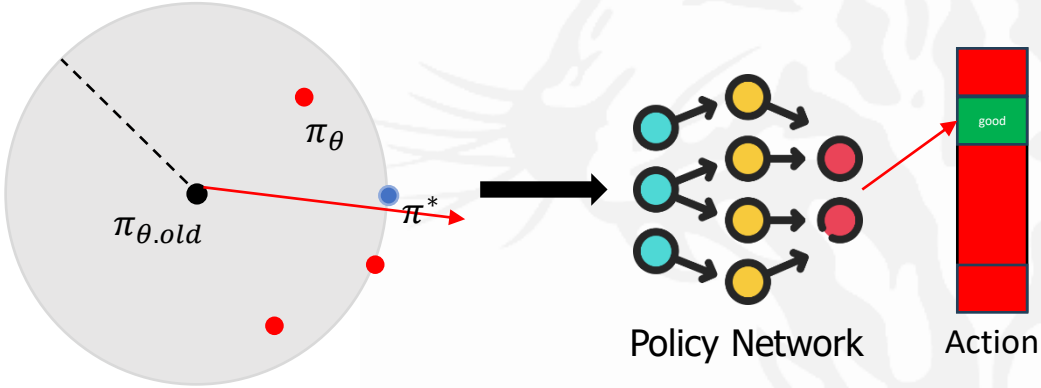
- **Background**
  - 불안정한 정책 기울기
    - 스텝 사이즈가 클 경우:
      - 불안정한 정책을 생성
      - 불안정한 샘플을 수집



    - 스텝 사이즈가 작을 경우:
      - 학습이 느림



- **Background**
  - 데이터 비효율성
    - On-policy method : 새로운 정책마다 완전히 새로운 궤적(trajectory)을 생성 필요
    - 데이터는 단 한 번의 기울기 업데이트 후 버려짐
    - 복잡한 신경망은 많은 업데이트가 필요하므로, 학습 과정이 매우 느려짐



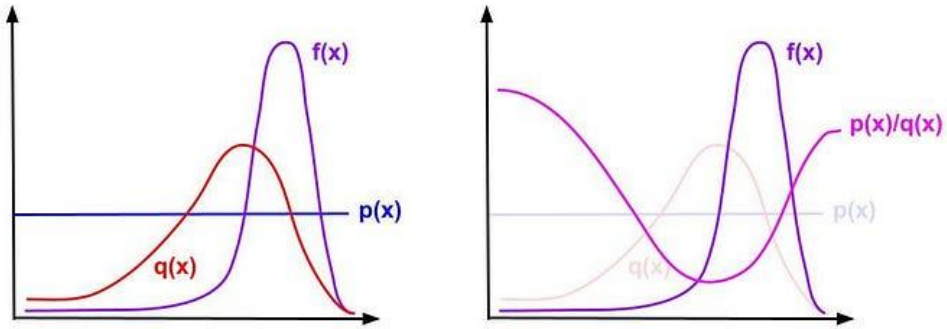
- **Idea of PPO**
  - Importance Sampling
    - $\max_\theta E_t\left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}\hat{A}_t\right], s.t. \hat{\mathrm{E}}_t\left[KL\left[\pi_{\theta_{old}}(.|s_t), \pi_\theta(.|s_t)\right]\right] \leq \delta$
  - Clipping
    - $L^{CLIP}(\theta) = \hat{E}_t\left[\min\left(r_t(\theta)\hat{A}_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right] = \hat{E}_t\left[r_t(\theta)\hat{A}_t\right]$
  - Generalized Advantage Estimate
    - $\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \cdots + (\gamma\lambda)^{T-t+1}\delta_{T-1}$
    - $where\ \delta_t = r_t + \gamma V(s_t + 1) - V(s_t)$

- **Idea of PPO**
  - Importance Sampling
    - 하나의 분포에서 샘플링하여 다른 분포를 추정하는 방법
    - Importance sampling은 새로운 분포$q(x)$를 도입하여, $f(x)$의 기대값을 더 효율적으로 추정함.
    - $E_{x\sim p}[f(x)] \approx \frac{1}{N}\sum_{i=1,x^i\in p}^{N} f(x^i)$
    - $E_{x\sim p}[f(x)] = \int f(x)p(x)dx$

$$= \int f(x)\frac{p(x)}{q(x)}q(x)dx$$

$$= E_{x\sim p}\left[f(x)\frac{p(x)}{q(x)}\right]$$

$$\approx \frac{1}{N}\sum_{i=1,x^i\in q}^{N} f(x^i)\frac{p(x^i)}{q(x^i)}$$



- **Idea of PPO**
  - Importance Sampling on PPO
    - 데이터 재사용
    - 에피소드가 끝난 후 결과를 반영하는 대신, 학습 과정에서 단계별(Step-by-Step)으로 데이터 반영
    - PPO는 과거 정책에서 수집한 데이터를 현재 정책 학습에 활용함:

$$\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$

    - 이 비율을 통하여 현재 정책이 과거 정책보다 특정 행동을 더 선호하거나 덜 선호하는 정도를 나타냄.
    - 1보다 커질수록 현재 정핵이 과거보다 해당 행동을 더 선호함.
    - 1보다 작을수록 현재 정책이 과거보다 해당 행동을 덜 선호함.

    - **이를 통하여 이전 정책에서 수집한 경험 데이터를 현재 정책 학습에 반영할 수 있어 경험 재활용이 가능.**

PPO 이론 및 설계

- **Idea of PPO**
  - Importance Sampling
    - $\max_\theta E_t\left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}\hat{A}_t\right], s.t. \hat{\mathrm{E}}_t\left[KL\left[\pi_{\theta_{old}}(.|s_t), \pi_\theta(.|s_t)\right]\right] \leq \delta$
  - Clipping
    - $L^{CLIP}(\theta) = \hat{E}_t\left[\min\left(r_t(\theta)\hat{A}_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right] = \hat{E}_t\left[r_t(\theta)\hat{A}_t\right]$
  - Generalized Advantage Estimate
    - $\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \cdots + (\gamma\lambda)^{T-t+1}\delta_{T-1}$
    - $where\ \delta_t = r_t + \gamma V(s_t + 1) - V(s_t)$

- **Idea of PPO**
  - PPO는 클리핑을 통하여 이전 정채과 현재 정책 간의 변화량을 제한함.
  - 이를 통해 정책의 급격한 변경을 방지하고 학습을 안정적으로 유지함.

  - KL Divergence
    - 두 확률 분포 간의 차이(거리)를 측정하는 지표

$$D_{KL}(p||q) = \sum_{x} p(x) \log_2 \frac{p(x)}{q(x)}$$

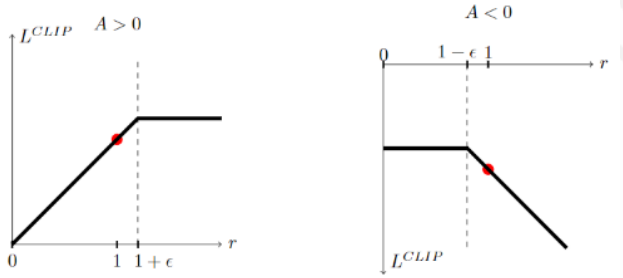
    - PPO에서는 KL Divergence를 이용하여 이전 정책과 현재 정책 간의 차이를 측정할 수 있음.

$$D_{KL}(\pi_1||\pi_2) = \sum_{a \in A} \pi_1(a|s) \log_2 \frac{\pi_1(a|s)}{\pi_2(a|s)}$$

- **Idea of PPO**
  - PPO는 **Clipping** 기법을 사용하여 이전 정책과 현재 정책 간의 거리를 제한하며 정책을 개선함.
  - **Clipped Objective**
    - PPO는 TRPO의 **KL 발산 제약**을 Clipping **함수**로 간소화하여 계산 복잡성을 줄이고 효율성을 높임.

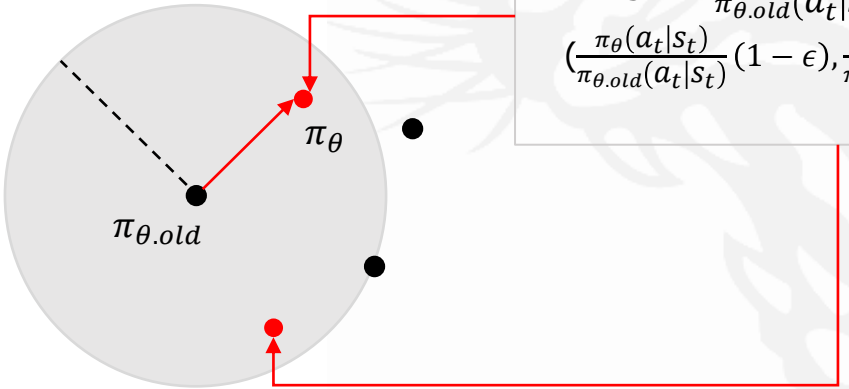
$$L^{CLIP}(\theta) = E_t[\min(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta.old}(a_t|s_t)}\hat{A}_t, clip(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta.old}(a_t|s_t)}, 1-\epsilon, 1+\epsilon)\hat{A}_t]$$

    - 중요도 비율기반으로 정책을 학습하며, Clipping을 통해 과도한 업데이트를 방지



- **Idea of PPO**
  - PPO는 이전 정책과 현재 정책 간의 **거리**를 제한하여 안정적으로 정책을 개선함.

  - **Clipped Objective**
    - Clipping은 $\pi_\theta(a_t|s_t)$을 해당 범위로 제한하여 $\hat{A}_t$ 에 과도한 영향을 주지 않도록 제약
    - $\hat{A}_t$ 가 발산하거나, 과도한 학습으로 인하여 성능이 급격히 저하되는 것을 방지



Policies that <u>guarantee</u>

Change in $\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta.old}(a_t|s_t)}\hat{A}_t$ for range of

$\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta.old}(a_t|s_t)}(1-\epsilon), \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta.old}(a_t|s_t)}(1+\epsilon)\right)$

- **Idea of PPO**
  - Importance Sampling
    - $\max_\theta E_t\left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}\hat{A}_t\right], s.t. \hat{\mathrm{E}}_t\left[KL\left[\pi_{\theta_{old}}(.|s_t), \pi_\theta(.|s_t)\right]\right] \leq \delta$
  - Clipping
    - $L^{CLIP}(\theta) = \hat{E}_t\left[\min\left(r_t(\theta)\hat{A}_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right] = \hat{E}_t\left[r_t(\theta)\hat{A}_t\right]$
  - Generalized Advantage Estimate
    - $\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \cdots + (\gamma\lambda)^{T-t+1}\delta_{T-1}$
    - $where\ \delta_t = r_t + \gamma V(s_t + 1) - V(s_t)$

- **Idea of PPO**
  - Generalized Advantage Estimate
    - Advantage function을 추정하는 방법
    - 기본적인 Advantage function:

$$A_t = R_t + \gamma V(s_{t+1}) - V(s_t)$$

    - R_t 스텝 t 에서의 보상, $V(s_t)$ 상태 t에서의 가치 함수, γ 할인율(Discount function)

    - GAE는 여러 시간 단계의 보상과 가치를 고려하여 이점(advantage)을 추정하는 다단계(multi-step) 추정 방식
    - GAE function:

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \cdots + (\gamma\lambda)^{T-t+1}\delta_{T-1}$$
$$where\ \delta_t = r_t + \gamma V(s_t + 1) - V(s_t)$$

    - δ_t Time Differential 오차로, λ 는 GAE에서 편향과 분산 간의 균형을 조절하는 하이퍼파라미터
    - GAE는 다단계 추정을 통하여 보상과 가치 정보를 보다 안정적으로 통합함.

- State

$$S = \{S_t, S_{t-1}, S_{WP}, S_{PA}, S_{IA}, S_{Tar}\}$$

(Agent 위치, Agent 이전위치, 변침점 위치 집합, Impact Angle 기반 변침점 위치, 목표물 위치)

$$S_t = \{x_t, y_t, z_t\}$$

$$S_{t-1} = \{x_{t-1}\ y_{t-1}, z_{t-1}\}$$

$$S_{WP} = \{(x_1, y_1, z_1), \dots, (x_N, y_N, z_N)\}$$

$$S_{PA} = \{(x_1, y_1, z_1, r_1), \dots, (x_M, y_M, z_M, r_M)\}$$

$$S_{Tar} = \{x_{Tar}, y_{Tar}, z_{Tar}\}$$

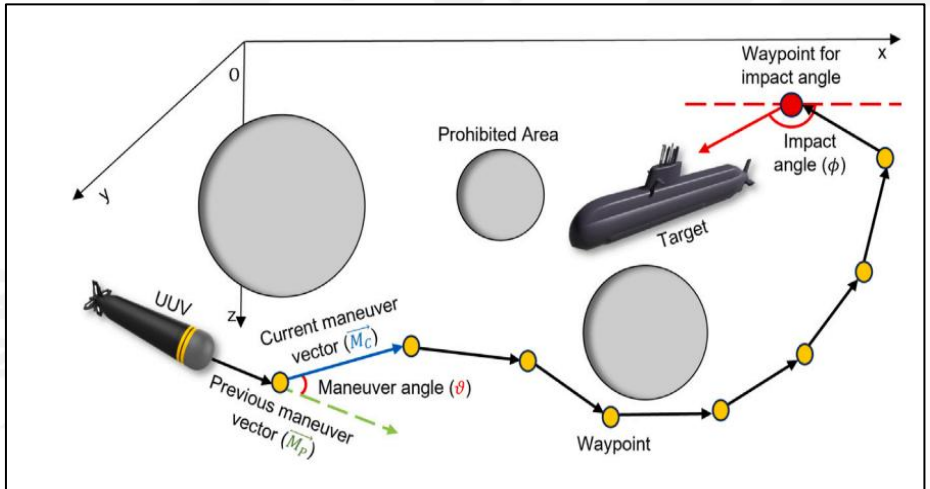


Fig. 1. Waypoint based mission-oriented UUV control towards target in complex 3D maritime environment.

- Action

$$A = \{\vartheta_t, \psi_t\}$$

(시간 t에서의 Yaw Angle, 시간 t에서의 Pitch Angle)

제약 조건)
최대 기동 각도: 90도
Step 별 이동 거리: 구의 반지름
다음 State: 행동에 해당하는 구 위의 한 점

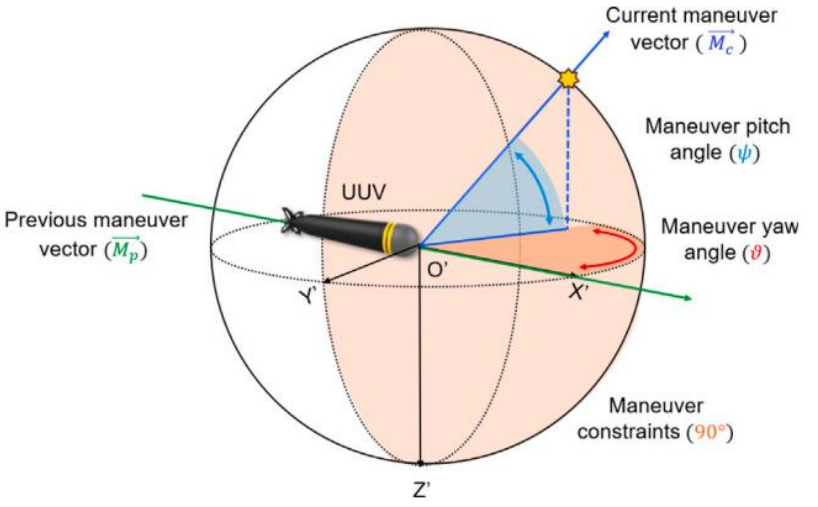


Fig. 3. Action space of UUV according to the maximum rotation angle.

수정된 PPO기반 무인이동체 제어 개요

- Reward

$$R_1 = -a + \sqrt{a^2(K^2 - n_t^2)}$$

$$R_2 = \begin{cases} -1 \\ R_1 \end{cases}$$

$$R_3 = \begin{cases} 1 & \text{if mission success} \\ 0 & \text{Otherwise} \end{cases}$$

$$R = 10R_3 + R_2(1 - R_3) - d$$

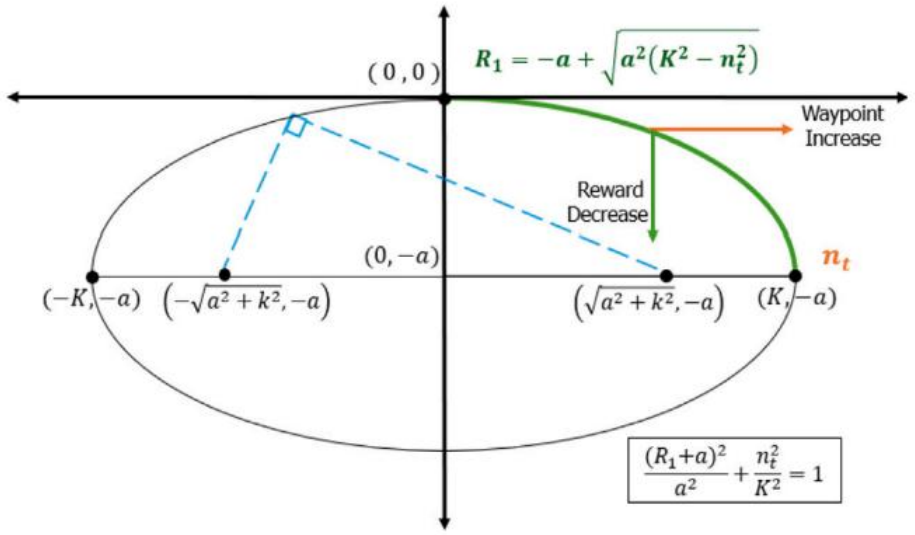


Fig. 4. The design of the reward function.

PPO 이론 및 설계

- PPO 설계 구조도

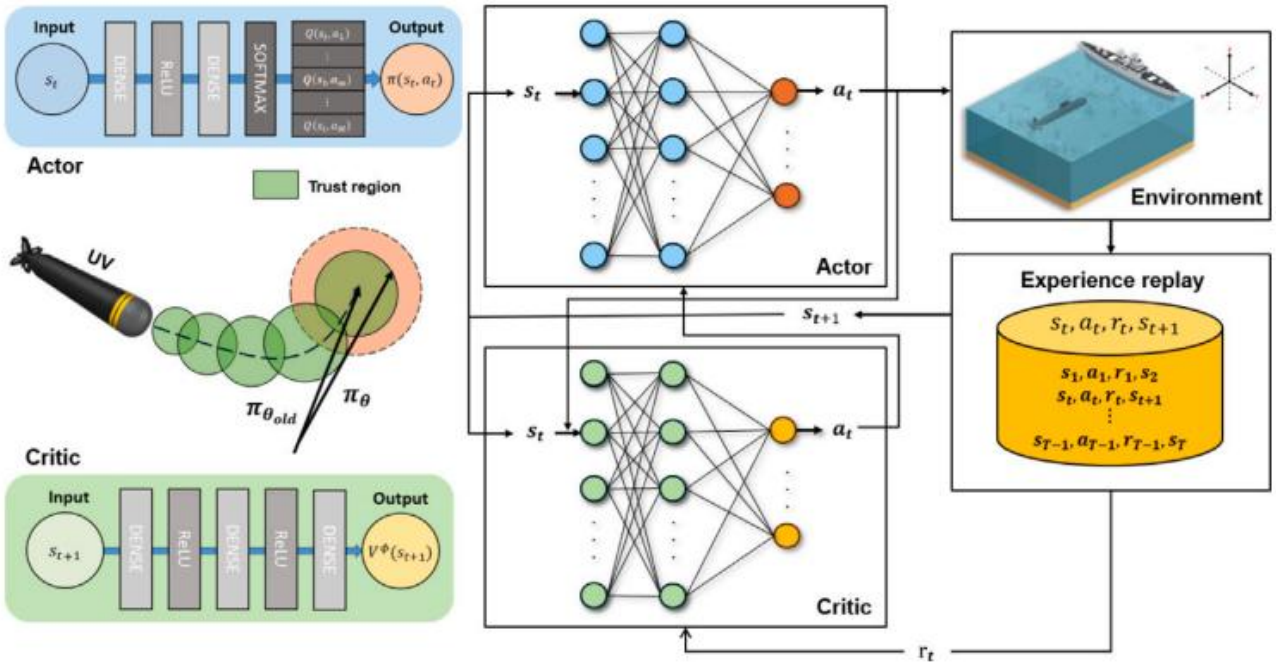


Fig. 5. The network structure diagram of DPO algorithm within the actor–critic framework.

- 수정된 PPO기반 무인이동체 제어 결과

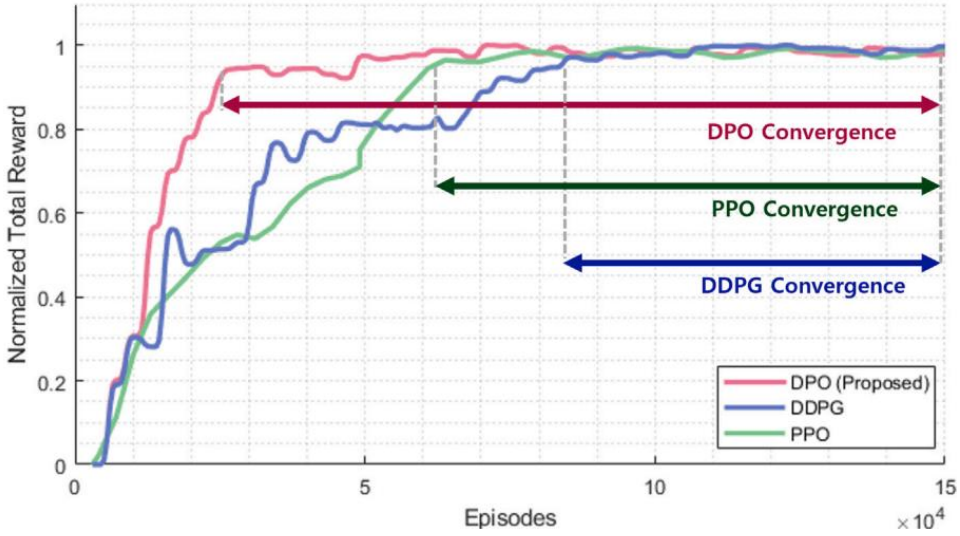


Fig. 7. Normalized total rewards per 150,000 episodes.

보상함수 수렴성

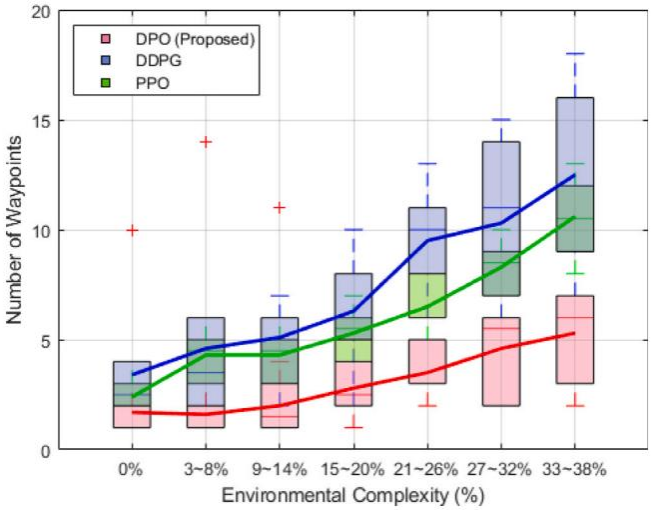


Fig. 8. Number of generated waypoints according to environment complexity.

변침점 생성 개수

- 수정된 PPO기반 무인이동체 제어 시각화 결과

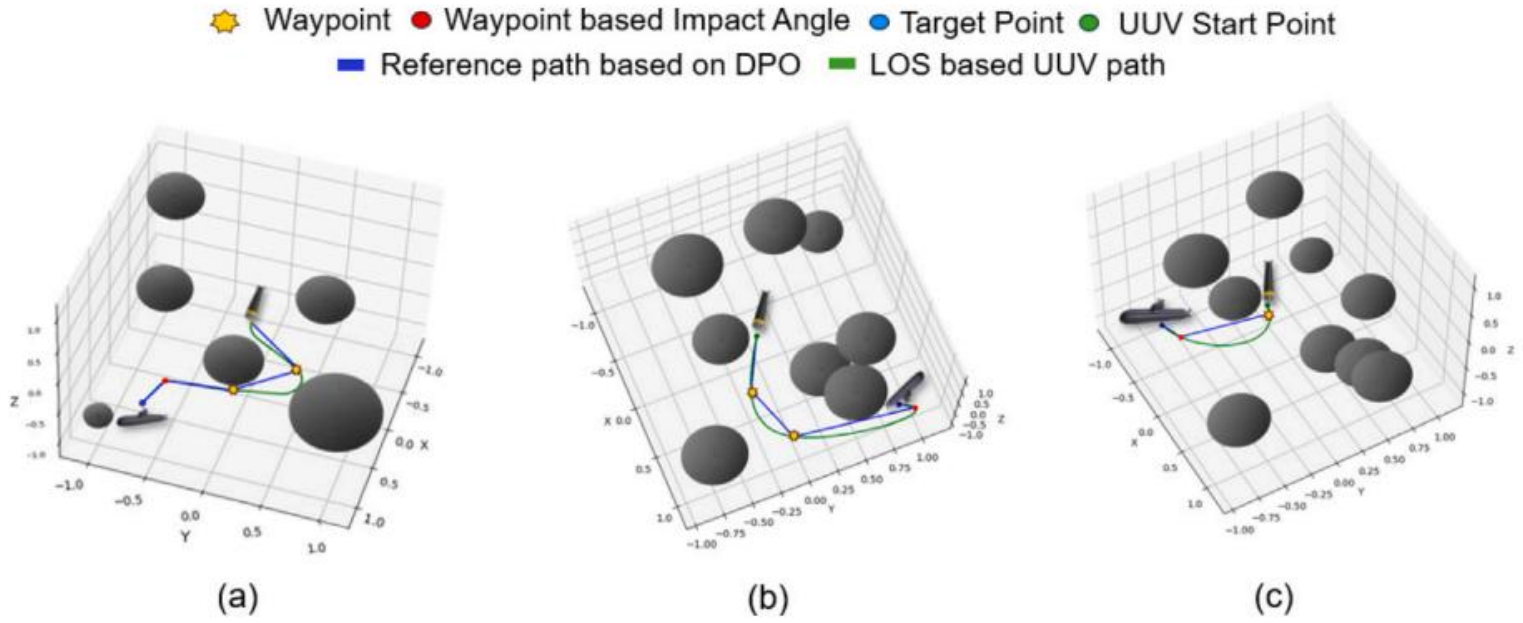


Fig. 10. Visualization results of proposed DPO based mission-oriented UUV control.

목차

수정된 PPO기반 무인이동체 제어 연구 개요

수정된 PPO 이론 및 설계

수정된 PPO기반 무인이동체 제어 구현

Q&A

수정된 PPO기반 무인이동체 제어 구현


```
1  import numpy as np
2  import torch
3  import torch.nn as nn
4  import torch.optim as optim
5  from torch.distributions import Categorical
6  import matplotlib.pyplot as plt
7  import os
8  import random
9  seed = 2025
10 deterministic = True
11
12 random.seed(seed)
13 np.random.seed(seed)
14 torch.manual_seed(seed)
15 torch.cuda.manual_seed_all(seed)
16 if deterministic:
17   torch.backends.cudnn.deterministic = True
18   torch.backends.cudnn.benchmark = False
```

[Line 1] **Numpy** Library 수치 계산 및 배열 연산 등을 위함.
[Line 2] **torch** library 딥러닝 프레임워크로 신경망 구축과 학습을 위함.
[Line 3] **torch.nn** 신경망 모듈로 레이어 정의 및 구성을 위함.
[Line 4] **torch.optim**학습률, 가중치 업데이트 등 제어를 위함.
[Line 5] **torch.distributions** 행동 샘플링 및 확률 계산을 위함.
[Line 6] **matplotlib.pyplot** 학습 결과 및 환경 시각화를 위함.
[Line 7] **os** 파일 경론 및 시스템 작업(모델 저장 등)을 위함.
[Line 8] **random** 환경에서의 난수 생성을 위함.
[Line 9] **seed** 값 설정, 난수 생성기가 항상 동일한 결과를 생성하도록 함.
[Line 10] **deterministic** PyTorch 연산을 재현할 수 있도록 설정함.
[Lines 11-14] 난수 생성기 초기화: Python, NumPy, PyTorch의 난수 시드를 동일하게 설정함.
[Lines 15-16] **PyTorch** 설정: GPU 연산이 결정론적이며 일관되게 실행되도록 설정함.

수정된 PPO기반 무인이동체 제어 구현


```
# Define the custom grid environment with obstacles
class GridEnvironment:
    def __init__(self, grid_size=6, goal=(5, 5)):
        self.grid_size = grid_size
        self.goal = goal
        self.reset()

    def reset(self):
        self.agent_pos = [0, 0]
        self.last_action = None
        self.obstacles = self._generate_obstacles()
        return self._get_state()

    def _generate_obstacles(self):
        obstacles = set()
        while len(obstacles) < 3:  # Generate 2 rand
            obs = (np.random.randint(0, self.grid_size), np.random.randint(0, self.grid_size))
            if obs != tuple(self.agent_pos) and obs != self.goal:
                obstacles.add(obs)
        return obstacles
```

[Line 18] 그리드 환경 클래스 설정: 강화학습 에이전트를 학습시키기 위함.
[Line 19] 초기화 함수 격자 크기와 목표 위치를 설정함.
[Lines 20-22] 환경의 초기 상태를 정의함.
[Line 23] **환경 초기화 함수:** 에이전트를 초기 위치로 재설정함.
[Line 24] 에이전트의 초기 위치를 (0,0)으로 설정함.
[Line 25] 이전 행동을 저장하기 위한 변수.
[Line 26] 장애물 생성을 위하여 함수 호출함.
[Line 27] 학습을 진행할 현재 환경을 반환함.

수정된 PPO기반 무인이동체 제어 구현


```
# Define the custom grid environment with obstacles
class GridEnvironment:
    def __init__(self, grid_size=6, goal=(5, 5)):
        self.grid_size = grid_size
        self.goal = goal
        self.reset()

    def reset(self):
        self.agent_pos = [0, 0]
        self.last_action = None
        self.obstacles = self._generate_obstacles()
        return self._get_state()

    def _generate_obstacles(self):
        obstacles = set()
        while len(obstacles) < 3:  # Generate 2 random obstacles
            obs = (np.random.randint(0, self.grid_size), np.random.randint(0, self.grid_size))
            if obs != tuple(self.agent_pos) and obs != self.goal:
                obstacles.add(obs)
        return obstacles
```

[Line 28] **장애물 생성 함수:** 그리드 내 무작위 위치에 장애물을 배치함.
[Lines 29-30] 장애물 개수 설정 및 생성함.
[Line 31] 랜덤으로 장애물의 위치를 생성하여 동적환경을 구성함.
[Line 32-33] 장애물이 에이전트의 초기 위치와 목표 위치에 설정되지 않도록 제약조건을 설정함.
[Line 34] 생성된 장애물의 위치를 반환함.

수정된 PPO기반 무인이동체 제어 구현


```
35 def _get_state(self):
36     state = np.zeros((self.grid_size, self.grid_size))
37     state[self.agent_pos[0], self.agent_pos[1]] = 1
38     for obs in self.obstacles:
39         state[obs[0], obs[1]] = -1
40     return state.flatten()
```

```
def step(self, action):
    # Define actions: 0=up, 1=down, 2=left, 3=right
    next_pos = self.agent_pos.copy()
    if action == 0 and self.agent_pos[0] >
        next_pos[0] -= 1
    elif action == 1 and self.agent_pos[0]
        next_pos[0] += 1
    elif action == 2 and self.agent_pos[1]
        next_pos[1] -= 1
    elif action == 3 and self.agent_pos[1] < self.grid_size - 1:
        next_pos[1] += 1
```

[Line 35] **환경 상태 반환을 위한 함수 정의**
[Line 36] 환경의 상태 배열을 생성함.
[Line 37] 에이전트의 위치를 1로 반영함.
[Lines 38-39] 장애물의 위치를 -1로 반영함.
[Line 40] 상태를 flatten 하여 반환함.

수정된 PPO기반 무인이동체 제어 구현


```
def step(self, action):
    # Define actions: 0=up, 1=down, 2=left, 3=right
    next_pos = self.agent_pos.copy()
    if action == 0 and self.agent_pos[0] > 0:
        next_pos[0] -= 1
    elif action == 1 and self.agent_pos[0] < self.grid_size - 1:
        next_pos[0] += 1
    elif action == 2 and self.agent_pos[1] > 0:
        next_pos[1] -= 1
    elif action == 3 and self.agent_pos[1] < self.grid_size - 1:
        next_pos[1] += 1
```

```
    # Check for collisions with obstacles
    if tuple(next_pos) in self.obstacles:
        reward = -1  # Penalty for hitting an ob
        done = True
    else:
        self.agent_pos = next_pos
        reward = 1 if tuple(self.agent_pos) == s
        if self.last_action is not None and self
            reward -= 0.1  # Penalty for changin

        done = tuple(self.agent_pos) == self.goa
```

[Line 41] **행동 처리 함수:** 에이전트가 주어진 행동을 수행하며, 상태, 보상, 종료 여부를 반환함.
[Line 42-53] 그리드 환경에서 에이전트가 수행할 수 있는 행동을 정의함.

0 : 위로 이동 (y축 +1)
1 : 아래로 이동 (y축 -1)
2 : 왼쪽으로 이동 (x축 -1)
3 : 오른쪽으로 이동 (x축 +1)

제약조건: 그리드 환경 사이즈에 맞도록 정의

수정된 PPO기반 무인이동체 제어 구현


```
        next_pos[0] += 1
    elif action == 2 and self.agent_pos[1] >
        next_pos[1] -= 1
    elif action == 3 and self.agent_pos[1] <
        next_pos[1] += 1

    # Check for collisions with obstacles
    if tuple(next_pos) in self.obstacles:
        reward = -1  # Penalty for hitting a
        done = True
    else:
        self.agent_pos = next_pos
        reward = 1 if tuple(self.agent_pos) == self.goal else -0.1
        if self.last_action is not None and self.last_action != action:
            reward -= 0.1  # Penalty for changing direction

        done = tuple(self.agent_pos) == self.goal

    self.last_action = action
    return self._get_state(), reward, done
```

[Lines 56-58] **장애물 충돌 확인:** 이동하려는 위치(next_pos)가 장애물에 해당하면 보상을 -1로 설정 후 에피소드 종료함.
[Lines 58-59] 장애물에 충돌하지 않으면 에이전트의 위치를 이동 위치로 업데이트함.
[Line 60] 목표 도달 확인: 목표 위치에 도달하면 보상을 +1 로 설정하고 , 그렇지 않으면 이동에 따라 -0.1의 페널티를 부여함.
[Line 61] 종료 조건 확인: 에이전트가 목표 위치에 도달하면 에피소드를 종료함.
[Line 62] **마지막 행동 저장:** 현재 행동을 저장하여 이전 행동을 저장함.
[Line 63] 결과 반환: 현재 상태(_get_state), 계산된 보상, 종료 여부 반환함.

수정된 PPO기반 무인이동체 제어 구현


```
# Define the policy network
class PolicyNetwork(nn.Module):
    def __init__(self, state_dim, action_dim):
        super(PolicyNetwork, self).__init__()
        self.fc = nn.Sequential(
            nn.Linear(state_dim, 128),
            nn.ReLU(),
            nn.Linear(128, action_dim),
        )

    def forward(self, x):
        return self.fc(x)

# PPO Agent
class PPOAgent:
    def __init__(self, state_dim, action_dim, lr=1e-3
        self.policy = PolicyNetwork(state_dim, action_
        self.optimizer = optim.Adam(self.policy.parame
        self.gamma = gamma
        self.clip_epsilon = clip_epsilon

    def select_action(self, state):
        state = torch.FloatTensor(state)
        logits = self.policy(state)
        distribution = Categorical(logits=logits)
        action = distribution.sample()
        return action.item(), distribution.log_prob(ac
```

[Line 64] **정책 신경망 정의:** PyTorch의 nn.Module을 상속받아 정의함.
[Line 65] **초기화 함수:** 입력 크기(state_dim)과 출력 크기(action_dim)을 인자로 받아 처리함.
[Line 66] 상위 클래스 초기화: nn.Module의 초기화 함수를 호출함.
[Lines 67-71] **신경망 정의:**
- state_dim -> 128 -> action_dim 구조의 완전 연결 신경망.
- ReLU 활성화 함수를 통하여 비선형성 추가.

[Line 72] 순전파 함수: 입력 데이터 처리를 위함.
[Line 73] 출력 계산: 신경망의 출력을 반환함.

수정된 PPO기반 무인이동체 제어 구현


```
# Define the policy network
class PolicyNetwork(nn.Module):
    def __init__(self, state_dim, action_dim):
        super(PolicyNetwork, self).__init__()
        self.fc = nn.Sequential(
            nn.Linear(state_dim, 128),
            nn.ReLU(),
            nn.Linear(128, action_dim),
        )

    def forward(self, x):
        return self.fc(x)

# PPO Agent
class PPOAgent:
    def __init__(self, state_dim, action_dim, lr=1e-3, gamma=0.99, clip_epsilon=0.2):
        self.policy = PolicyNetwork(state_dim, action_dim)
        self.optimizer = optim.Adam(self.policy.parameters(), lr=lr)
        self.gamma = gamma
        self.clip_epsilon = clip_epsilon

    def select_action(self, state):
        state = torch.FloatTensor(state)
        logits = self.policy(state)
        distribution = Categorical(logits=logits)
        action = distribution.sample()
        return action.item(), distribution.log_prob(action)
```

[Line 75] **PPO 에이전트 정의:** PPO 알고리즘을 사용하는 에이전트 클래스
[Line 76] **초기화 함수:** 상태 크기, 행동 크기, 학습률(lr), 할인율(gamma), 클리핑 파라미터(clip_epsilon)을 초기화함.
[Line 77] 정책 신경망 초기화: PolicyNetwork 인스턴스를 생성함.
[Line 78] **Adam 옵티마이저 설정: 정책 신경망의 가중치를 업데이트할 Adam 정의**
[Lines 79-80] 강화학습 파라미터 gamma와 PPO 클리핑 파라미터 설정함.

수정된 PPO기반 무인이동체 제어 구현


```
# Define the policy network
class PolicyNetwork(nn.Module):
    def __init__(self, state_dim, action_dim):
        super(PolicyNetwork, self).__init__()
        self.fc = nn.Sequential(
            nn.Linear(state_dim, 128),
            nn.ReLU(),
            nn.Linear(128, action_dim),
        )

    def forward(self, x):
        return self.fc(x)

# PPO Agent
class PPOAgent:
    def __init__(self, state_dim, action_dim, lr=1e-3, gamma=0.99, clip_epsilon=0.2):
        self.policy = PolicyNetwork(state_dim, action_dim)
        self.optimizer = optim.Adam(self.policy.parameters(), lr=lr)
        self.gamma = gamma
        self.clip_epsilon = clip_epsilon

    def select_action(self, state):
        state = torch.FloatTensor(state)
        logits = self.policy(state)
        distribution = Categorical(logits=logits)
        action = distribution.sample()
        return action.item(), distribution.log_prob(action)
```

[Line 82] **행동 선택 함수:** 현재 상태를 기반으로 행동 샘플링 함.
[Line 83] **상태 변환:** 상태 데이터를 PyTorch 텐서로 변환함.
[Line 84] **정책 출력 계산:** 신경망의 출력, 각 행동의 로짓(logit) 값을 계산함.
[Line 85] **범주형 분포 생성:** 로짓 값을 기반으로 행동의 확률 분포 생성
[Line 86] **행동 샘플링:** 확률 분포에서 행동을 샘플링함.
[Line 87] **행동 및 로그 확률 반환**: 샘플링된 행동과 해당 로그 확률을 반환함.

수정된 PPO기반 무인이동체 제어 구현


```
89 def update(self, trajectories):
90     states, actions, rewards, old_log_probs = zip(*trajectories)
91     states = torch.FloatTensor(states)
92     actions = torch.LongTensor(actions)
93     rewards = torch.FloatTensor(rewards)
94     old_log_probs = torch.stack(old_log_probs)
95
96     # Compute returns
97     returns = []
98     G = 0
99     for reward in reversed(rewards):
100         G = reward + self.gamma * G
101         returns.insert(0, G)
102     returns = torch.FloatTensor(returns)
```

[Line 89] **업데이트 함수:** 수집한 데이터(trajectories)를 사용해 신경망을 업데이트함.
[Line 90] **데이터 분리:** 수집된 데이터에서 상태, 행동, 보상, 이전 로그 확률을 각각 분리함.
[Line 91-95] 데이터 변환: 데이터를 PyTorch 텐서로 변환함.
[Lines 97-102] **리턴 계산:** 역방향으로 누적 보상을 계산하여 리턴 값을 생성함.

수정된 PPO기반 무인이동체 제어 구현


```
    for _ in range(10):  # Update for multiple epochs
        logits = self.policy(states)
        distribution = Categorical(logits=logits)
        log_probs = distribution.log_prob(actions)

        # Detach returns and old_log_probs to prevent reusing the graph
        ratios = torch.exp(log_probs - old_log_probs.detach())
        advantages = returns.detach() - returns.mean()
        surr1 = ratios * advantages
        surr2 = torch.clamp(ratios, 1 - self.clip_ep

        loss = -torch.min(surr1, surr2).mean()

        self.optimizer.zero_grad()
        loss.backward()  # Backpropagation
        self.optimizer.step()

def save_model(self, path):
    torch.save(self.policy.state_dict(), path)

def load_model(self, path):
    self.policy.load_state_dict(torch.load(path))
```

[Lines 104-107] **로그 확률 계산:** 현재 정책의 로그 확률을 계산함.
[Line 110] **확률 비율 계산:** 새로운 정책과 이전 정책의 행동 확률 비율을 계산함.

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta old}(a_t|s_t)}$$

ratio = torch.exp(log_probs - old_log_probs)

[Line 111] **이점 계산:** 리턴 값에서 평균을 빼 이점을 계산
[Lines 112-113] **리턴 계산:** 역방향으로 누적 보상을 계산하여 리턴 값을 생성함.

$$\hat{A}_t = (\gamma\lambda)\delta_{t+1} + \ldots + (\gamma\lambda)^{T-t+1}\delta_{T-1} + \delta_t$$

advantage = gamma * lmbda * advantage + delta_t[0]

수정된 PPO기반 무인이동체 제어 구현


```
    for _ in range(10):  # Update for multiple epochs
        logits = self.policy(states)
        distribution = Categorical(logits=logits)
        log_probs = distribution.log_prob(actions)

        # Detach returns and old_log_probs to prevent reusing the graph
        ratios = torch.exp(log_probs - old_log_probs.detach())
        advantages = returns.detach() - returns.mean()
        surr1 = ratios * advantages
        surr2 = torch.clamp(ratios, 1 - self.clip_epsilon, 1 + self.clip_epsilon) * advantages

        loss = -torch.min(surr1, surr2).mean()

        self.optimizer.zero_grad()
        loss.backward()  # Backpropagation
        self.optimizer.step()

def save_model(self, path):
    torch.save(self.policy.state_dict(), path)

def load_model(self, path):
    self.policy.load_state_dict(torch.load(path))
```

[Lines 114-117] **로그 확률 계산:** 현재 정책의 로그 확률을 계산함.

$$L_t^{CLIP}(\theta) = \hat{\mathbb{E}}_t\left[\min\left(r_t(\theta)\hat{A}_t, clip(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t\right)\right]$$

surr1 = ratio * advantage

surr2 = torch.clamp(ratio, 1-eps_clip, 1+eps_clip) * advantage

[Lines 119-121] 역전파를 통해 가중치를 업데이트.
[Lines 123-124] **모델 저장:** 정책 신경망의 가중치를 파일로 저장.
[Lines 125-126] **모델 로드:** 저장된 가중치를 정책 신경망에 로드.

수정된 PPO기반 무인이동체 제어 구현


```
# Training loop
def train():
    env = GridEnvironment()
    state_dim = env.grid_size ** 2
    action_dim = 4
    agent = PPOAgent(state_dim, action_dim)
    model_path = "ppo_policy.pth"
    rewards_history = []

    for episode in range(1000):
        state = env.reset()
        done = False
        episode_rewards = 0
        trajectory = []

        while not done:
            action, log_prob = agent.select_actio
            next_state, reward, done = env.step(a

            trajectory.append((state, action, reward, log_prob))
            state = next_state
            episode_rewards += reward
```

[Line 128] **학습 함수 정의:** 에이전트를 학습시키고 학습된 모델을 반환하는 함수
[Line 129] **환경 생성:** 강화학습 에이전트가 작동할 그리드 환경 생성
[Line 130] **상태 공간 크기 계산:** 격자 환경의 모든 칸 수를 계산
[Line 131] **행동 공간 크기 설정:** 4가지 행동(상, 하, 좌, 우)으로 고정
[Line 132] **PPO 에이전트 초기화:** 상태와 행동 공간 크기를 사용해 PPO 에이전트를 생성
[Line 133] **모델 저장 경로 설정:** 학습된 모델을 저장할 파일 경로 설정
[Line 134] **보상 기록 초기화:** 에피소드별 총 보상을 저장할 리스트 초기화

수정된 PPO기반 무인이동체 제어 구현


```
# Training loop
def train():
    env = GridEnvironment()
    state_dim = env.grid_size ** 2
    action_dim = 4
    agent = PPOAgent(state_dim, action_dim)
    model_path = "ppo_policy.pth"
    rewards_history = []

    for episode in range(1000):
        state = env.reset()
        done = False
        episode_rewards = 0
        trajectory = []

        while not done:
            action, log_prob = agent.select_actio
            next_state, reward, done = env.step(a

            trajectory.append((state, action, rew
            state = next_state
            episode_rewards += reward
```

[Line 136] **에피소드 반복:** 1000번의 에피소드 동안 에이전트를 학습 설정
[Line 137] **환경 초기화:** 각 에피소드 시작 시 환경을 초기화하고 초기 상태
[Line 138-140] 학습 변수 초기화
done = 에피소드 종료 여부
episode_rewards = 현재 에피소드의 총 보상 누적
Trajectory = 에이전트의 경험 데이터 저장

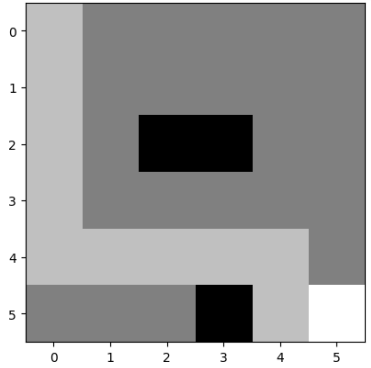
수정된 PPO기반 무인이동체 제어 구현

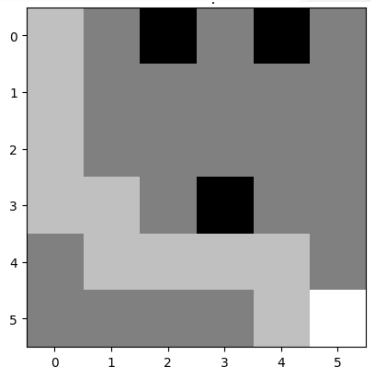

```
        while not done:
            action, log_prob = agent.select_action(state)
            next_state, reward, done = env.step(action)

            trajectory.append((state, action, reward, log_prob))
            state = next_state
            episode_rewards += reward

        rewards_history.append(episode_rewards)
        agent.update(trajectory)
        print(f"Episode {episode}: Total Reward = {episode_rewards}")

        if (episode + 1) % 100 == 0:
            agent.save_model(model_path)


    print("Training complete. Model saved.")
    return agent, env

if __name__ == "__main__":
    trained_agent, env = train()
    print("Visualizing policy...")
    visualize_policy(env, trained_agent)
```

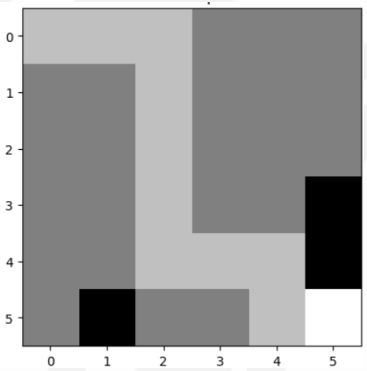
[Line 142] **에피소드 내 루프:** 에피소드 내 루프: 에피소드가 종료될 때까지 반복
[Line 143] **행동 선택:** 현재 상태를 기반으로 행동을 샘플링하고 로그 확률을 반환
[Line 144] **경험 저장:** 현재 상태, 행동, 보상, 로그 확률 trajectory에 추가
[Lines 146-147] **상태 및 보상 업데이트:** 다음 상태로 전환 및 총 보상에 현재 보상 누적
[Line 149] **보상 기록 업데이트**: 현재 에피소드의 총 보상을 저장
[Line 150] **정책 업데이트:** 수집된 trajectory 기반 정책 업데이트
[Line 151] **진행 상황 출력:** 현재 에피소드와 총 보상을 출력
[Line 153-154] **모델 저장:** 100에피소드마다 정책 신경망 저장
[Line 156-157] **결과 반환:** 학습된 에이전트와 환경을 반환

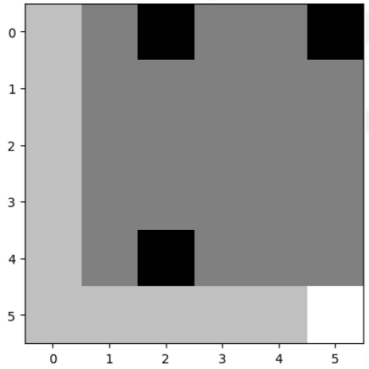
수정된 PPO기반 무인이동체 제어 구현


```
Episode 0: Total Reward = -1
Episode 1: Total Reward = -3.4000000000000017
Episode 2: Total Reward = -9.600000000000001
Episode 3: Total Reward = -5.200000000000002
Episode 4: Total Reward = -7.700000000000001
Episode 5: Total Reward = -3.3000000000000003
Episode 6: Total Reward = -4.100000000000001
Episode 7: Total Reward = -3.1
Episode 8: Total Reward = -1
Episode 9: Total Reward = -1
Episode 10: Total Reward = -2.9000000000000004
Episode 11: Total Reward = -2.8999999999999995
Episode 12: Total Reward = -3.0000000000000018
Episode 13: Total Reward = -1.1
Episode 14: Total Reward = -1.9
Episode 15: Total Reward = -3.1
Episode 16: Total Reward = -2.1
Episode 17: Total Reward = -4.0
Episode 18: Total Reward = -1.9
Episode 19: Total Reward = -1.4
Episode 20: Total Reward = -13.399999999999983
Episode 21: Total Reward = -2.2
Episode 22: Total Reward = -4.4
Episode 23: Total Reward = -2.4000000000000012
Episode 24: Total Reward = -4.500000000000003
Episode 25: Total Reward = -2.9000000000000004
Episode 26: Total Reward = -1.7
Episode 27: Total Reward = -1.2
Episode 28: Total Reward = -12.899999999999979
Episode 29: Total Reward = -1.3
Episode 30: Total Reward = -1.9
Episode 31: Total Reward = -7.900000000000003
Episode 32: Total Reward = -1.5
Episode 33: Total Reward = -5.999999999999999
```









https://colab.research.google.com/drive/1J1jf7B1kwlSPtPGI7QBRELzA_qRsSE30?usp=sharing

목차

수정된 PPO기반 무인이동체 제어 연구 개요

수정된 PPO 이론 및 설계

수정된 PPO기반 무인이동체 제어 구현

Q&A

Thank you for your attention!

- More questions?
  - joongheon@korea.ac.kr (Advisor)
  - emilyjroh@korea.ac.kr (Presenter)